

Conseal: Problem 3 (Fixing the Tool's Mistakes)

What I built

Sam's failure mode isn't a lack of detection — it's that he's moving fast and trusts the tool a little too much, so the mistakes that matter are the ones he never stops to look at. Conseal is built around that: a hybrid detection pipeline feeds a frontend whose entire job is to make corrections cheap and to force attention onto the misses Sam is most likely to skip.

Detection is layered deliberately, not just for accuracy but for what each layer is allowed to be trusted with. A regex pass catches structured PII (phone numbers, SSNs, IBANs, IPs) instantly and with exact offsets — this is the only layer treated as "system-critical," because it's the only one I can guarantee is correct. Everything ambiguous (names, orgs, addresses) goes through Gemini, but never the whole document at once: when a user flags an unclear term, only a tight context window around it is sent for classification, which keeps latency and cost down and — more importantly — keeps the AI's judgment scoped to exactly the sentence a human is already looking at.

The correction UX is the actual deliverable. X-Ray mode shows suggestions as translucent highlights rather than immediate black boxes, so Sam can read the document in context instead of taking redactions on faith. A tripwire surfaces any system-critical (regex-confirmed) entity the AI scored low on or missed, and blocks finalization until he's addressed it — this exists specifically because Sam's bias is to trust the tool, so the one layer guaranteed to be correct is the one allowed to interrupt him. Corrections propagate: marking a false positive safe, or flagging a missed name, uses KMP string matching to sync that decision across every occurrence in the document in one action, because asking Sam to repeat the same correction by hand is exactly the kind of friction that causes the eleventh instance of a missed name to slip through.

What I chose not to build, and why

I did not build full-document LLM scanning as the primary detection method. It's slower, more expensive, and — critically — gives Sam no reason to trust any individual flag more than another, which works against the actual problem: a tool he can interrogate, not one he takes on faith. The regex/AI split exists so some flags can be backed by certainty and others explicitly cannot be.

I did not build the confidence dashboard as a core feature, even though it's implemented. It answers "why should I trust this tool," which is Problem 1's question, not Problem 3's. I kept it as a closing summary rather than the centerpiece, so it didn't pull effort away from the correction loop itself.

I did not implement re-detection after manual edits, or support for editing the underlying document text. Redaction state lives entirely as an annotation layer over an immutable

source string — this keeps X-Ray mode, propagation, and final export all provably consistent views of the same data, at the cost of not supporting in-place text editing, which wasn't part of the brief.

I did not build multi-document batch handling (Problem 2's concern). Scoping to a single-document correction loop, done well, mattered more than a shallower version that tried to cover both problems.